

Summen Zahid

(B00996747)

COM772 - Emerging and Advanced Topics in AI

-

Explainable AI for Mental Health Classification

Code Document

"""

Explainable AI for mental health treatment prediction.

This script:

- builds a synthetic mental-health related dataset**
- trains several classifiers and compares performance**
- explains the best model with SHAP and LIME**
- analyses fairness across gender**
- applies a simple bias-mitigation strategy**
- generates plots and a short ethical report**

"""

```
import os
```

```
import warnings
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import pandas as pd
```

```
import seaborn as sns
```

```
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import (
```

```
    accuracy_score,
```

```
    confusion_matrix,
```

```
    f1_score,
```

```

precision_score,
recall_score,
roc_auc_score,
)

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier

warnings.filterwarnings("ignore")

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
np.random.seed(42)
plt.style.use("seaborn-v0_8-darkgrid")
sns.set_palette("husl")

def load_data(n_samples: int = 1000) -> pd.DataFrame:
    """Create a synthetic mental-health style dataset."""
    genders = ["Male", "Female", "Other"]
    countries = ["United States", "United Kingdom", "Canada", "Germany", "Other"]

    data = {
        "Age": np.random.randint(18, 70, n_samples),
        "Gender": np.random.choice(genders, n_samples, p=[0.7, 0.25, 0.05]),
        "Country": np.random.choice(countries, n_samples, p=[0.4, 0.2, 0.15, 0.1, 0.15]),
        "self_employed": np.random.choice(["Yes", "No"], n_samples, p=[0.2, 0.8]),
        "family_history": np.random.choice(["Yes", "No"], n_samples, p=[0.4, 0.6]),
        "work_interfere": np.random.choice(
            ["Never", "Rarely", "Sometimes", "Often"],

```

```

    n_samples,
    p=[0.2, 0.3, 0.35, 0.15],
),
"no_employees": np.random.choice(
    ["1-5", "6-25", "26-100", "100-500", "More than 1000"],
    n_samples,
    p=[0.2, 0.25, 0.25, 0.15, 0.15],
),
"remote_work": np.random.choice(["Yes", "No"], n_samples, p=[0.3, 0.7]),
"tech_company": np.random.choice(["Yes", "No"], n_samples, p=[0.7, 0.3]),
"benefits": np.random.choice(
    ["Yes", "No", "Don't know"], n_samples, p=[0.5, 0.3, 0.2]
),
"care_options": np.random.choice(
    ["Yes", "No", "Not sure"], n_samples, p=[0.4, 0.3, 0.3]
),
"wellness_program": np.random.choice(
    ["Yes", "No", "Don't know"], n_samples, p=[0.3, 0.5, 0.2]
),
"seek_help": np.random.choice(
    ["Yes", "No", "Don't know"], n_samples, p=[0.4, 0.4, 0.2]
),
}

```

```
df = pd.DataFrame(data)
```

```
# Create treatment label with some intuitive correlations + a small gender bias.
```

```
base_p = 0.3
```

```

probs = []

for _, row in df.iterrows():

    p = base_p

    if row["family_history"] == "Yes":

        p += 0.25

    if row["work_interfere"] in ["Often", "Sometimes"]:

        p += 0.2

    if row["benefits"] == "Yes":

        p += 0.15

    if row["Gender"] == "Female":

        p += 0.1

    probs.append(min(p, 0.9))

df["treatment"] = np.where(np.random.rand(len(df)) < np.array(probs), "Yes", "No")

return df

```

```

def explore_data(df: pd.DataFrame) -> None:

```

```

    """Quick EDA and basic plots."""

```

```

    print("Data shape:", df.shape)

```

```

    print("\nColumns:", list(df.columns))

```

```

    print("\nTarget distribution:\n", df["treatment"].value_counts(normalize=True))

```

```

    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

```

```

    df["treatment"].value_counts().plot(

```

```

        kind="bar", ax=axes[0], color=["#2ecc71", "#e74c3c"]

```

```

    )

```

```

    axes[0].set_title("Treatment seeking")

```

```
axes[0].set_xlabel("")
axes[0].set_ylabel("Count")
```

```
axes[1].hist(df["Age"], bins=20, color="#3498db", edgecolor="black", alpha=0.7)
axes[1].set_title("Age distribution")
axes[1].set_xlabel("Age")
```

```
plt.tight_layout()
path = os.path.join(BASE_DIR, "01_data_exploration.png")
plt.savefig(path, dpi=300, bbox_inches="tight")
plt.close()
print("Saved:", path)
```

```
def preprocess(df: pd.DataFrame):
    """Encode categorical variables, scale features and split X / y."""
    df = df.copy()
    X = df.drop("treatment", axis=1)
    y = df["treatment"]

    target_encoder = LabelEncoder()
    y_enc = target_encoder.fit_transform(y)

    encoders = {}
    for col in X.select_dtypes(include=["object"]).columns:
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col].astype(str))
        encoders[col] = le
```

```
feature_names = list(X.columns)
```

```
X_unscaled = X.copy()
```

```
scaler = StandardScaler()
```

```
X_scaled = pd.DataFrame(
```

```
    scaler.fit_transform(X), columns=feature_names, index=X.index
```

```
)
```

```
return X_scaled, y_enc, feature_names, encoders, target_encoder, scaler, X_unscaled
```

```
def train_models(X_train, X_test, y_train, y_test, feature_names):
```

```
    """Train a small model zoo and pick the best by F1 score."""
```

```
    models = {
```

```
        "Random Forest": RandomForestClassifier(
```

```
            n_estimators=100, max_depth=10, random_state=42
```

```
        ),
```

```
        "Gradient Boosting": GradientBoostingClassifier(
```

```
            n_estimators=100, max_depth=5, random_state=42
```

```
        ),
```

```
        "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
```

```
        "Decision Tree": DecisionTreeClassifier(max_depth=8, random_state=42),
```

```
    }
```

```
    results = {}
```

```
    for name, model in models.items():
```

```
        model.fit(X_train, y_train)
```

```
        y_pred = model.predict(X_test)
```

```
        y_proba = model.predict_proba(X_test)[:, 1]
```

```

metrics = {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred, average="weighted"),
    "recall": recall_score(y_test, y_pred, average="weighted"),
    "f1": f1_score(y_test, y_pred, average="weighted"),
    "auc": roc_auc_score(y_test, y_proba),
    "y_pred": y_pred,
    "y_proba": y_proba,
    "model": model,
}

results[name] = metrics

```

```

best_name = max(results, key=lambda k: results[k]["f1"])
best_model = results[best_name]["model"]
print(f"Best model: {best_name} (F1={results[best_name]['f1']:.3f})")

```

Simple performance bar chart.

```

perf = pd.DataFrame(
    {
        name: {
            "Accuracy": r["accuracy"],
            "Precision": r["precision"],
            "Recall": r["recall"],
            "F1": r["f1"],
        }
        for name, r in results.items()
    }
)

```


).T

```
fig, ax = plt.subplots(figsize=(10, 5))
perf.plot(kind="bar", ax=ax)
ax.set_ylim(0, 1.05)
ax.set_title("Model comparison")
plt.tight_layout()
path = os.path.join(BASE_DIR, "02_model_comparison.png")
plt.savefig(path, dpi=300, bbox_inches="tight")
plt.close()
print("Saved:", path)
```

```
# Confusion matrix for best model.
cm = confusion_matrix(y_test, results[best_name]["y_pred"])
fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False,
    xticklabels=["No", "Yes"],
    yticklabels=["No", "Yes"],
    ax=ax,
)
ax.set_title(f"Confusion matrix – {best_name}")
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
```

```

plt.tight_layout()

path = os.path.join(BASE_DIR, "03_confusion_matrix.png")

plt.savefig(path, dpi=300, bbox_inches="tight")

plt.close()

print("Saved:", path)

# Feature importance for tree-based models.

if hasattr(best_model, "feature_importances_"):
    importances = best_model.feature_importances_
    idx = np.argsort(importances)[::-1][:15]

fig, ax = plt.subplots(figsize=(10, 6))
ax.barh(range(len(idx)), importances[idx], color="#3498db")
ax.set_yticks(range(len(idx)))
ax.set_yticklabels([feature_names[i] for i in idx])
ax.invert_yaxis()
ax.set_title(f"Top features – {best_name}")
plt.tight_layout()

path = os.path.join(BASE_DIR, "04_feature_importance.png")

plt.savefig(path, dpi=300, bbox_inches="tight")

plt.close()

print("Saved:", path)

return results, best_model, best_name

def run_shap(model, X_test, feature_names, model_name):
    """Compute SHAP values and save a few standard plots."""
    try:

```

```

import shap
except ImportError:
    print("Installing shap...")
    import subprocess

    subprocess.check_call(["pip", "install", "shap", "-q"])
    import shap # type: ignore

if any(k in model_name for k in ["Random Forest", "Decision Tree", "Gradient"]):
    explainer = shap.TreeExplainer(model)
    values = explainer.shap_values(X_test)
    if isinstance(values, list):
        values = values[1]
else:
    explainer = shap.KernelExplainer(model.predict_proba, shap.sample(X_test, 100))
    values = explainer.shap_values(X_test[:100])
    if isinstance(values, list):
        values = values[1]

plt.figure(figsize=(10, 6))

shap.summary_plot(values, X_test, feature_names=feature_names, plot_type="bar",
show=False)

path = os.path.join(BASE_DIR, "05_shap_summary.png")
plt.tight_layout()
plt.savefig(path, dpi=300, bbox_inches="tight")
plt.close()
print("Saved:", path)

```

```

plt.figure(figsize=(10, 8))

shap.summary_plot(values, X_test, feature_names=feature_names, show=False)

path = os.path.join(BASE_DIR, "06_shap_detailed.png")

plt.tight_layout()

plt.savefig(path, dpi=300, bbox_inches="tight")

plt.close()

print("Saved:", path)


shap.force_plot(
    explainer.expected_value if not isinstance(explainer.expected_value, list) else
explainer.expected_value[1],
    values[0:1],
    X_test.iloc[0:1],
    feature_names=feature_names,
    matplotlib=True,
    show=False,
)

path = os.path.join(BASE_DIR, "07_shap_force.png")

plt.tight_layout()

plt.savefig(path, dpi=300, bbox_inches="tight")

plt.close()

print("Saved:", path)


return values, explainer


def run_lime(model, X_train, X_test, feature_names):

    """Generate a few LIME explanations and save as a grid of bar plots."""

    try:

```

```

import lime

import lime.lime_tabular

except ImportError:

    print("Installing lime...")

    import subprocess

    subprocess.check_call(["pip", "install", "lime", "-q"])

    import lime # type: ignore

    import lime.lime_tabular # type: ignore

explainer = lime.lime_tabular.LimeTabularExplainer(

    X_train.values,

    feature_names=feature_names,

    class_names=["No treatment", "Treatment"],

    mode="classification",

    random_state=42,

)

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

axes = axes.ravel()

for i in range(4):

    idx = i * 25

    exp = explainer.explain_instance(

        X_test.iloc[idx].values, model.predict_proba, num_features=10

    )

    pairs = exp.as_list()

    labels = [p[0] for p in pairs]

    vals = [p[1] for p in pairs]

```

```
colors = ["#2ecc71" if v > 0 else "#e74c3c" for v in vals]
```

```
axes[i].barh(range(len(labels)), vals, color=colors)
```

```
axes[i].set_yticks(range(len(labels)))
```

```
axes[i].set_yticklabels(labels, fontsize=8)
```

```
axes[i].axvline(0, color="black", linewidth=1)
```

```
axes[i].set_title(f"Sample {idx}")
```

```
plt.tight_layout()
```

```
path = os.path.join(BASE_DIR, "08_lime_explanations.png")
```

```
plt.savefig(path, dpi=300, bbox_inches="tight")
```

```
plt.close()
```

```
print("Saved:", path)
```

```
return explainer
```

```
def gender_fairness(df, model, X_test, y_test, feature_names, encoders,  
X_test_unscaled):
```

```
    """Compute simple fairness metrics by gender and save a summary chart."""
```

```
    y_pred = model.predict(X_test)
```

```
    X_ref = pd.DataFrame(X_test_unscaled, columns=feature_names)
```

```
    gender_encoder = encoders["Gender"]
```

```
    X_ref["gender"] = X_ref["Gender"].apply(  
        lambda x: gender_encoder.inverse_transform([int(x)])[0]
```

```
    )
```

```
    X_ref["y"] = y_test
```

```
    X_ref["y_pred"] = y_pred
```

```

metrics = {}

for g in X_ref["gender"].unique():

    sub = X_ref[X_ref["gender"] == g]

    if len(sub) == 0:

        continue

    pos_rate = (sub["y_pred"] == 1).mean()


    tp = ((sub["y"] == 1) & (sub["y_pred"] == 1)).sum()
    fp = ((sub["y"] == 0) & (sub["y_pred"] == 1)).sum()
    pos = (sub["y"] == 1).sum()
    neg = (sub["y"] == 0).sum()


    tpr = tp / pos if pos else 0
    fpr = fp / neg if neg else 0
    prec = tp / (sub["y_pred"] == 1).sum() if (sub["y_pred"] == 1).sum() else 0


    metrics[g] = {

        "pos_rate": pos_rate,

        "tpr": tpr,

        "fpr": fpr,

        "precision": prec,

        "count": len(sub),

    }


df_m = pd.DataFrame(metrics).T


fig, axes = plt.subplots(2, 2, figsize=(12, 8))

```

```
df_m["pos_rate"].plot(kind="bar", ax=axes[0, 0], color="#3498db")
axes[0, 0].set_title("Positive prediction rate")
```

```
df_m["tpr"].plot(kind="bar", ax=axes[0, 1], color="#2ecc71")
axes[0, 1].set_title("True positive rate")
```

```
df_m["fpr"].plot(kind="bar", ax=axes[1, 0], color="#e74c3c")
axes[1, 0].set_title("False positive rate")
```

```
df_m["precision"].plot(kind="bar", ax=axes[1, 1], color="#9b59b6")
axes[1, 1].set_title("Precision")
```

```
plt.tight_layout()
path = os.path.join(BASE_DIR, "09_fairness_analysis.png")
plt.savefig(path, dpi=300, bbox_inches="tight")
plt.close()
print("Saved:", path)
```

```
return metrics
```

```
def mitigate_bias(X, y, feature_names, encoders, X_unscaled):
```

```
    """Re-weight samples to reduce gender imbalance in positive predictions."""
```

```
    idx = np.arange(len(X))
```

```
    train_idx, test_idx = train_test_split(
```

```
        idx, test_size=0.2, random_state=42, stratify=y
```

```
    )
```

```
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
```



```
X_train_u, X_test_u = X_unscaled.iloc[train_idx], X_unscaled.iloc[test_idx]
```

```
y_train, y_test = y[train_idx], y[test_idx]
```

```
X_train_df = pd.DataFrame(X_train_u, columns=feature_names)
```

```
weights = np.ones(len(X_train))
```

```
for g_val in X_train_df["Gender"].unique():
```

```
    mask_g = X_train_df["Gender"] == g_val
```

```
    mask_pos = y_train == 1
```

```
    n = ((mask_g) & (mask_pos)).sum()
```

```
    if n > 0:
```

```
        w = len(y_train) / (2 * n)
```

```
        weights[np.where(mask_g.values & mask_pos)[0]] = w
```

```
rf_weighted = RandomForestClassifier(
```

```
    n_estimators=100, max_depth=10, random_state=42
```

```
)
```

```
rf_weighted.fit(X_train, y_train, sample_weight=weights)
```

```
rf_plain = RandomForestClassifier(
```

```
    n_estimators=100, max_depth=10, random_state=42
```

```
)
```

```
rf_plain.fit(X_train, y_train)
```

```
y_pred_plain = rf_plain.predict(X_test)
```

```
y_pred_weighted = rf_weighted.predict(X_test)
```

```
print(
```

```

"Original RF F1:",
f1_score(y_test, y_pred_plain, average="weighted"),
" | Weighted RF F1:",
f1_score(y_test, y_pred_weighted, average="weighted"),
)

```

```

# Compare positive prediction rates by gender.

```

```

X_test_df = pd.DataFrame(X_test_u, columns=feature_names)
gender_encoder = encoders["Gender"]

```

```

comparison = {}

```

```

for name, preds in [("Original", y_pred_plain), ("Weighted", y_pred_weighted)]:

```

```

    X_test_df["pred"] = preds

```

```

    rates = {}

```

```

    for g_val in X_test_df["Gender"].unique():

```

```

        g = gender_encoder.inverse_transform([int(g_val)])[0]

```

```

        m = X_test_df["Gender"] == g_val

```

```

        rates[g] = (X_test_df.loc[m, "pred"] == 1).mean()

```

```

    comparison[name] = rates

```

```

comp_df = pd.DataFrame(comparison)

```

```

fig, ax = plt.subplots(figsize=(8, 5))

```

```

comp_df.plot(kind="bar", ax=ax)

```

```

ax.set_ylabel("Positive prediction rate")

```

```

ax.set_title("Effect of re-weighting on gender fairness")

```

```

plt.tight_layout()

```

```

path = os.path.join(BASE_DIR, "10_bias_mitigation.png")

```

```

plt.savefig(path, dpi=300, bbox_inches="tight")

```

```
plt.close()
```

```
print("Saved:", path)
```

```
return rf_weighted, comparison
```

```
def write_ethics_report(fairness_before_after, gender_metrics):
```

```
    """Write a short text file summarising fairness and ethics points."""
```

```
    lines = []
```

```
    lines.append("Ethical analysis – mental health prediction model\n")
```

```
    lines.append("Fairness by gender (before mitigation):\n")
```

```
    for g, m in gender_metrics.items():
```

```
        lines.append(
```

```
            f"- {g}: pos_rate={m['pos_rate']:.3f}, "
```

```
            f"TPR={m['tpr']:.3f}, FPR={m['fpr']:.3f}, "
```

```
            f"precision={m['precision']:.3f}, n={m['count']}"
```

```
        )
```

```
    lines.append("\nEffect of simple re-weighting:\n")
```

```
    for model_name, rates in fairness_before_after.items():
```

```
        lines.append(f"{model_name}:")
```

```
        for g, r in rates.items():
```

```
            lines.append(f" - {g}: {r:.3f}")
```

```
    lines.append(
```

```
        "\nKey points:\n"
```

```
        "- The model can exhibit different error rates across gender groups.\n"
```

```
        "- Re-weighting trades a little performance for better balance.\n"
```

```
        "- Any deployment should include human oversight and continuous monitoring.\n"
```

```
)
```

```
path = os.path.join(BASE_DIR, "ethical_report.txt")
```

```
with open(path, "w") as f:
```

```
    f.write("\n".join(lines))
```

```
print("Saved:", path)
```

```
def main():
```

```
    print("Loading data...")
```

```
    df = load_data()
```

```
    print("Exploring data...")
```

```
    explore_data(df)
```

```
    print("Preprocessing...")
```

```
    X, y, feature_names, encoders, target_enc, scaler, X_unscaled = preprocess(df)
```

```
    X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y, test_size=0.2, random_state=42, stratify=y
```

```
)
```

```
    X_train_u, X_test_u = train_test_split(
```

```
        X_unscaled, test_size=0.2, random_state=42, stratify=y
```

```
)[0:2]
```

```
    print("Training models...")
```

```
    _, best_model, best_name = train_models(
```

```
        X_train, X_test, y_train, y_test, feature_names
```

```
)
```

```
print("Running SHAP...")
```

```
run_shap(best_model, X_test, feature_names, best_name)
```

```
print("Running LIME...")
```

```
run_lime(best_model, X_train, X_test, feature_names)
```

```
print("Analysing fairness...")
```

```
gender_metrics = gender_fairness(
```

```
    df, best_model, X_test, y_test, feature_names, encoders, X_test_u  
)
```

```
print("Applying bias mitigation...")
```

```
_, fairness_comparison = mitigate_bias(
```

```
    X, y, feature_names, encoders, X_unscaled  
)
```

```
print("Writing ethical summary...")
```

```
write_ethics_report(fairness_comparison, gender_metrics)
```

```
print("Done.")
```

```
if __name__ == "__main__":
```

```
    main()
```